QFF26 UIQC Project Plan: SQD Pipeline

Team details

- **Team name:** Minq
- **Members:** Jehovah-nissi Njoagwuani and Michael Mokuolu
- **Forked repository:** github.com/HOVA63/qff26xUIQC

Project title

Sample-Based Quantum Diagonalization (SQD): Completing Challenge Notebooks 01 to 03

Brief description

We will complete the SQD challenge notebooks in the repository (`01_sample_then_diagonalize`, `02_molecules_as_bitstrings,` and `03_the_full_sqd_pipeline`), working through all of their exercises and checkpoints.

SQD samples bitstrings from a quantum circuit, crops the molecular Hamiltonian to the sampled configurations, and diagonalizes it classically. We will build the pipeline step by step across the three notebooks, from a toy magnet model to H2 to stretched N2, and make sure both of us can explain every stage.

SQD Project: Presentation, Methodology, Results and Discussion

1. Project presentation

Sample-based quantum diagonalization (SQD) finds a system's ground-state energy by letting a quantum circuit propose which basis states matter, then letting a classical computer solve the problem only inside that small shortlist. This project works through SQD in three stages, from a toy spin chain to a stretched N_2 molecule, and tests one claim: a circuit that is a poor approximation of the ground state can still give an accurate energy.

Why it matters. The Hamiltonian of an n-qubit system is a $2^n \times 2^n$ matrix. Storing a single 50-qubit state needs about 18 petabytes, so exact diagonalization stops being possible at roughly molecule-sized problems. Ground-state energies decide whether a drug molecule binds or how a battery electrode holds charge, so this wall is a real obstacle.

The idea in one line. Ground states are often concentrated on a few bitstrings. Measure a circuit, keep the distinct bitstrings it returns, crop the Hamiltonian to those rows and columns, and diagonalize the small block. The result is variational: it can only approach the true energy from above, and adding bitstrings can only improve it.

Questions tested.

1. Does a sampled shortlist beat a random shortlist of the same size?
2. Does sample-and-diagonalize beat the circuit's own energy estimate when the circuit is deliberately crude?
3. On a hard molecule (N_2 stretched to 1.5 Å), does the full pipeline reach chemical accuracy (1.6 mHa), and does it survive noisy samples?

2. Methodology

All work was done in Python with Qiskit 2.5.1 on classical simulators (NumPy for exact diagonalization, Qiskit Aer for sampling). No real quantum hardware was used. The project follows three notebooks of increasing difficulty.

Stage 1: spin chain (8 qubits). The model is a transverse-field Ising chain, a row of tiny magnets with a sideways field of strength g:

H = -\sum_{i} Z_i Z_{i+1} - g \sum_i X_i
Exact diagonalization of the 256 × 256 matrix at g = 0.4 gave the reference energy. I then (a) measured how concentrated the exact ground state is, (b) implemented subspace_energy, which crops H to a set of bitstrings and returns the lowest eigenvalue of the block, (c) compared nested lists sampled from the exact ground state against 100 random nested lists of equal size,

and (d) built a deliberately crude circuit (a GHZ ladder plus an RY twist of angle θ on every qubit, no optimization) and compared its own expectation-value energy with the energy from sampling it and diagonalizing the distinct bitstrings.

Stage 2: molecules as bitstrings (H_2). Each bitstring is split into alpha-spin and beta-spin electron seats, so one bitstring is an electron configuration. PySCF supplies the one- and two-electron integrals (hcore, eri), and solve_fermion performs the crop-and-diagonalize step. The key detail is that the solver works on the product of the distinct alpha strings and the distinct beta strings, so two sampled rows can span a 4-determinant space. I tracked energy against subspace size for H_2 and across stretched bond lengths, where Hartree-Fock fails.

Stage 3: full pipeline (N_2 at 1.5 Å). The triple bond is stretched to a strongly correlated geometry (STO-3G basis, 2 core orbitals frozen, 10 electrons in 8 orbitals, 16 qubits). The pipeline has four steps:

1. **Integrals and references:** PySCF computes Hartree-Fock, CCSD, and exact FCI energies; FCI is the answer key.
2. **Ansatz:** a UCJ circuit from ffsim, initialized from the CCSD amplitudes with 2 repetitions. It conserves electron number, so every noiseless shot is a valid configuration (3,136 of 65,536 bitstrings, under 5%).
3. **Sampling:** the circuit is compiled and sampled for 20,000 shots on Aer.
4. **Configuration recovery and diagonalization:** diagonalize_fermionic_hamiltonian from qiskit-addon-sqd repairs invalid rows using orbital occupancies and re-solves, for up to 6 passes (3 batches of 500 samples).

To test robustness, I added a toy noise model, flipping each recorded bit independently with probability 2% (and later 5%) after sampling. A controlled four-condition experiment changed only two factors, sample type and number of passes:

Condition	Samples	Recovery passes
A	clean	1
B	clean	6
C	2% bit-flip noise	1
D	2% bit-flip noise	6

Finally, the pipeline was reused through a single run_sqd() call on water (STO-3G, 1 frozen core orbital, 12 qubits) as an independent example.

3. Results

All numbers below are the outputs recorded in the notebooks (seeded runs, simulator only).

Stage 1: a crude circuit still gives a good energy. The exact ground-state energy at g = 0.4 is −7.405591. Sampling the exact state 800 times produced only 27 distinct bitstrings out of 256, and the energy error of the nested sampled lists fell monotonically, as the variational guarantee requires. For the crude circuit at θ = 0.35, the circuit's own energy was off by 1.23, while sampling it and diagonalizing the 36 distinct bitstrings cut the error to 3.7×10^{-2}, about 33 times better.

Twist θ	Distinct bitstrings (1,024 shots)	Circuit's own error	Sample + diagonalize error
0.00	2	0.406	0.406
0.20	19	0.682	0.0494
0.35	36	1.229	0.0371
0.70	119	3.311	0.00539
1.00	199	5.362	0.00359
1.50	140	7.371	0.1995

The circuit's own error rises with θ, but the sampled-subspace error falls until θ ≈ 1 and then worsens at θ = 1.5, where the fixed shot budget is spread too thinly.

Stage 2: H_2 and stretched H_2. Hartree-Fock misses the exact H_2 energy by 24.92 mHa (−1.126755 vs −1.151673 Ha). The energy becomes exact once the sampled rows have supplied all 4 alpha and all 4 beta strings, which span all 16 valid determinants. At a stretched 2.5 Å bond, Hartree-Fock is off by 143.9 mHa, while a 4-determinant space built from only 2 sampled rows is off by 13.1 mHa.

Stage 3: N_2 at 1.5 Å (16 qubits). Reference errors against FCI (−107.581483 Ha): Hartree-Fock +309.03 mHa, CCSD +13.77 mHa. Chemical accuracy is 1.6 mHa.

Condition	Distinct sampled rows	Determinant dimension	Error vs FCI (mHa)
A: clean, 1 pass	97	306	+15.94
B: clean, 6 passes	97	306	+15.94
C: 2% noise, 1 pass	832	1,050	+4.26
D: 2% noise, 6 passes	832	2,021	+0.60

Of the 20,000 noiseless shots, 100% had valid electron counts. After 2% bit flips, 73.1% did (theory: 73.3%). At 5% noise, only 48% of rows stayed valid, yet six passes of recovery reached +0.10 mHa (dimension 2,862). On water (12 qubits), `run_sqd()` reached +0.04 mHa against FCI, compared with +49.48 mHa for Hartree-Fock and +0.12 mHa for CCSD.

4. Discussion

What the results show. The central claim held in all three stages. A circuit that is 1.23 energy units wrong on the spin chain still pointed to bitstrings that gave a 3.7×10^{-2} error after classical diagonalization. The reason is a division of labour: the circuit only needs to visit the right neighbourhood (its support), and the classical eigensolver reassigns every amplitude optimally. On N_2 at 1.5 Å, where Hartree-Fock is 309 mHa off and CCSD 13.8 mHa off, six-pass SQD reached +0.60 mHa, inside chemical accuracy. Subspace quality depends on which configurations are included, not only how many, and the θ sweep shows that more diverse samples help only until the shot budget is spread too thin.

Noise and recovery. Configuration recovery turned unusable rows into valid ones: 27% of 2%-noise rows had broken electron counts, and the pipeline repaired them instead of discarding them. The comparison C versus D isolates the benefit: with samples, batching, and seed fixed, extra passes cut the error from +4.26 to +0.60 mHa. On clean samples (A versus B), extra passes changed nothing, because the 306-determinant subspace had already stopped growing.

What should not be over-read.

- Condition D (noisy) ended below condition B (clean). This is one seeded run. Recovery can create valid configurations that were never measured, so it can enlarge the subspace, but this does not show that noise helps in general.
- The noise is a simple post-sampling bit-flip model. Real devices also have gate errors, correlated errors, and crosstalk, so these numbers are not a hardware prediction.
- Everything ran on classical simulators. N_2 in STO-3G (3,136 configurations) and water in 12 qubits are small enough to solve exactly, which is why FCI could serve as an answer key. The same workflow has been run on much larger problems on hardware, but that scaling was not tested here.
- Results come from one seed per setting; I did not measure run-to-run variance.
- On water, CCSD was already within 0.12 mHa, so SQD's gain there (0.04 mHa) is small. The method's value shows most clearly where CCSD struggles, as with stretched N_2.

Possible extensions. Stretch N_2 further (2.0 Å), where correlation is harder, raise the noise rate until recovery breaks, vary the shot count to find where sampling starves, test spin symmetrization (`symmetrize_spin=True`), and repeat with several seeds to put error bars on each result.

Attached is the GitHub repo: https://github.com/HOVA63/Team-Minq

Assistance Declaration

- We reached out to Dr. Sophie Engineer and Victor Adeyemi for guidance on choosing a project.
- We watched the following videos as recommended by Dr. Sophie Engineer to help decide what project to pick:
    - VQE - What Is the Variational Quantum Eigensolver? | VQE Explained
    - SQD - Faster Quantum Computing with Sample-based Algorithms | SQD in Action
    - QAOA - Solving MaxCut with QAOA: Quantum Tutorial for 100+ Qubits
- We used Google Gemini, Claude, and ChatGPT to break down the material for understanding.
- Google Gemini was used to explain broken code.
  Claude assisted in the creation of the project report.
- Grammarly was used for grammar correction in this project report.